

Merging IFC and USD

A strategic approach to integrating the semantic business logic of Industry Foundation Classes within the high-performance architecture of Universal Scene Description.

The Strategy

- The Industry Foundation Classes (IFC) schema excels at defining the strict semantic logic, relationships, and property data of buildings.
- Universal Scene Description (USD) provides an unparalleled, high-performance framework for composing massive 3D scenes.
- By migrating IFC definitions into USD native schemas, we map IFC's abstract structures to USD's core concepts.
- This enables rich building data to natively power advanced rendering, physics simulation, and real-time digital twins.

Core USD Concepts

- **class:** Establishes a reusable template. In IFC, this directly maps to abstract base definitions or `IfcTypeObject` elements, defining pure structural topology without a physical presence.
- **def:** Defines a concrete, physical instance. In IFC, this correlates to an `IfcProduct` occurrence instantiated within a specific level of the spatial hierarchy.
- **over:** An overriding declaration that retroactively augments an object. This is how we inject IFC semantics, specific property sets, and massive geometric arrays.

Understanding "class"

In USD, a `class` establishes a reusable blueprint without actually placing an object into the scene hierarchy.

In our IFC merging strategy, classes are leveraged to define the foundational definition of a building element—such as a generic `Wall`. This includes mapping out its necessary sub-components, such as its `Body` (`Mesh`), `Axis`, and `Directrix` (`BasisCurves`).

Think of it as the ultimate "Type" definition. It guarantees that every subsequent `Wall` instance will inherit this exact topological structure.

Understanding "def"

A `def` block creates a tangible, concrete instance in the USD scene. It moves a concept from a blueprint into physical space.

For our IFC model, this is the exact moment an abstract wall "class" becomes a physical occurrence. It is instantiated within a strict spatial container, mimicking the IFC Spatial Structure (e.g., inside a specific `IfcSpace` or `IfcBuildingStorey`).

The instance simply points to its parent class, inheriting all underlying traits and topologies instantly.

Understanding "over"

The `over` concept is the linchpin for merging IFC's heavy business logic with USD's lightweight scene structure.

Instead of cluttering the initial definition, USD allows us to non-destructively "override" or augment an existing `class` or `def` retroactively.

We use `over` declarations strictly to inject the semantic payload: designating the `ifc5:class`, assigning custom Property Sets (like external boundaries), linking materials, and streaming raw geometric vertex data.

IFC Naming Conventions & Suffixes

- In this schema, you will see base names like N93791d5d5beb437bb8ec2f1f0ba4bf3b and concatenated names like N93791d5d5beb437bb8ec2f1f0ba4bf3b_Body.
- The **base name** represents the top-level, abstract architectural container. It is the core IFC element, such as the overall Wall object.
- The **concatenated name** represents a specific geometric sub-component belonging to that parent element.
- This structure is necessary because a single IFC object might contain multiple distinct representations simultaneously, such as a physical 3D mesh (_Body), a centerline (_Axis), or a 2D footprint (_FootPrint).
- Appending text ensures unique identification for these sub-components within the USD hierarchy without naming collisions.

IFC to USD Pipeline

- **Step 1: class** - Establish the base IFC element templates and required sub-topologies.
- **Step 2: def** - Instantiate physical elements precisely within the USD spatial hierarchy.
- **Step 3: over (Logic)** - Retroactively inject business logic, IFC classes, and custom properties.
- **Step 4: over (Geom)** - Stream massive geometric coordinate arrays and complex materials.

Code Summary: Blueprint & Instantiation

1. Blueprint (class)

```
{  
  "def": "class",  
  "type": "UsdGeom:Xform",  
  "name": "N93791d..."  
}
```

2. Instantiation (def & inherits)

```
{  
  "def": "def",  
  "name": "Wall",  
  "inherits": ["</N93791...>"]  
}
```

Code Summary: Augmentation & Naming

3. Augmentation (over)

```
{  
  "def": "over",  
  "attributes": {  
    "ifc5:class": {"code": "IfcWall"}  
  }  
}
```

4. Name Expansion

```
{  
  "def": "def",  
  "name": "Body",  
  "inherits": ["</N9379..._Body>"]  
}
```